



Piscine Rust: Module 00

Summary: First steps and core mechanics of the Rust language.

Version: 1.00

Contents

I	Preamble	3
II	General Rules	5
III	Exercise 00: Initial Spark	6
IV	Exercise 01: Data Vault	7
V	Exercise 02: Fluid State	8
VI	Exercise 03: Variable Shadowing	9
VII	Exercise 04: Branch Paths	10
VIII	Exercise 05: Condition Loop	11
IX	Exercise 06: Range Loop	12
X	Exercise 07: Pattern Match	13
XI	Turn in and Submission	14

Chapter I

Preamble

Welcome to the first module of the Rust Piscine.

This module serves as an introduction to the `Rust` programming language. Rust is a modern systems programming language designed to provide both high performance and strong memory safety guarantees. It focuses on reliability, efficient resource management, and preventing common classes of programming errors at compile time.

The goal of this module is to help you become familiar with the fundamental concepts of the language.

Throughout this module, you will discover:

- the basic syntax of Rust;
- variables and immutability;
- functions;
- control flow structures;
- the type system;
- the first notions of ownership and borrowing.

Unlike many other programming languages, Rust introduces concepts that may initially seem unfamiliar. This piscine is designed to help you understand them progressively through practice. The exercises encourage experimentation and learning through implementation.

You are encouraged to carefully read the instructions, test your programs frequently, and consult the official documentation when necessary.

Learning Rust requires precision and attention to detail, but it provides powerful tools to write safe, efficient, and maintainable software.

Good luck and enjoy the Rust Piscine.



Take care of your ownership...

Chapter II

General Rules

- Your program should not crash in any circumstances and should not quit unexpectedly. If it happens, your project will be considered non-functional and your grade will be 0.
- Your code must be written in Rust 1.93+
- You must compile your programs with the following command to treat all warnings as errors, ensuring cleaner and safer code: `rustc -D warnings <file.rs>`.
- Your programs must respect clippy linter standards.
- Function names must match exactly what is requested.
- Cargo is forbidden only for this module. Only the rustc compiler is authorized.
- By Odin, by Thor! Use your brain!!!

Chapter III

Exercise 00: Initial Spark

	Exercise
Exercise 00: Initial Spark	
Turn-in directory: <i>ex00/</i>	
Files to turn in: <code>ft_hello.rs</code>	
Allowed functions: <code>print!()</code> , <code>println!()</code>	

Write a Rust program that prints the message `Hello, World!` followed by a newline.

Expected output:

```
$> ./ft_hello
Hello, World!
```

The output must match exactly the expected result, including capitalization and punctuation.



Be careful: any extra characters (spaces, additional text, missing newline, etc.) will cause the exercise to be considered incorrect.

Chapter IV

Exercise 01: Data Vault

	Exercise
Exercise 01: Data Vault	
Turn-in directory: <i>ex01/</i>	
Files to turn in: <code>ft_variables.rs</code>	
Allowed functions: <code>print!()</code> , <code>println!()</code>	

Write a program that declares a variable containing the value 42 and prints it to the standard output.

Expected output:

```
$> ./ft_variables
42
```



Rust variables are immutable by default. This means their value cannot be modified after initialization.

Chapter V

Exercise 02: Fluid State

	Exercise
Exercise 02: Fluid State	
Turn-in directory: <i>ex02/</i>	
Files to turn in: <code>ft_mut.rs</code>	
Allowed functions: <code>print!()</code> , <code>println!()</code>	



The goal of this exercise is to understand how mutable variables work in Rust.

In Rust, variables are immutable by default. If you want to modify the value of a variable after its creation, you must declare it as `mut`.

Write a program that:

- creates a variable with the value 5;
- prints its value;
- modifies the variable to contain 6;
- prints the new value.

Expected output:

```
$> ./ft_mut
5
6
```


Chapter VI

Exercise 03: Variable Shadowing

	Exercise
Exercise 03: Variable Shadowing	
Turn-in directory: <i>ex03/</i>	
Files to turn in: <code>ft_shadowing.rs</code>	
Allowed functions: <code>print!()</code> , <code>println!()</code>	

In Rust, you can declare a new variable with the same name as a previous variable. This is called *shadowing*. Unlike the `mut` keyword, shadowing allows you to change the type of the value while reusing the exact same variable name.

Write a program that:

- declares a variable containing the integer 42;
- prints its value;
- shadows this variable by declaring a new one with the same name, containing the string "forty-two";
- prints the new value.

Expected output:

```
$> ./ft_shadowing
42
forty-two
```

Chapter VII

Exercise 04: Branch Paths

	Exercise
Exercise 04: Branch Paths	
Turn-in directory: <i>ex04/</i>	
Files to turn in: <code>ft_branch.rs</code>	
Allowed functions: <code>print!()</code> , <code>println!()</code>	

Write a program that declares a variable `number` initialized to 0. Use conditional statements to evaluate this variable:

- if the number is strictly greater than 0, print `Positive`;
- if the number is strictly less than 0, print `Negative`;
- if the number is exactly 0, print `Zero`.

Expected output:

```
$> ./ft_branch
Zero
```

Chapter VIII

Exercise 05: Condition Loop

	Exercise
Exercise 05: Condition Loop	
Turn-in directory: <i>ex05/</i>	
Files to turn in: <code>ft_while.rs</code>	
Allowed functions: <code>print!()</code> , <code>println!()</code>	



Learn how to repeat actions using the **while** loop.

Write a program that uses a **while** loop to print a countdown. The program must start at 3, print each number down to 1, and finally print **Go!**.

Expected output:

```
$> ./ft_while
3
2
1
Go!
```

Chapter IX

Exercise 06: Range Loop

	Exercise
Exercise 06: Range Loop	
Turn-in directory: <i>ex06/</i>	
Files to turn in: <code>ft_for.rs</code>	
Allowed functions: <code>print!()</code> , <code>println!()</code>	



Discover the `for` loop and how to iterate over a sequence of numbers using ranges.

Write a program that uses a `for` loop to print all integers from 1 to 5 inclusive.

Expected output:

```
$> ./ft_for
1
2
3
4
5
```

Chapter X

Exercise 07: Pattern Match

	Exercise
Exercise 07: Pattern Match	
Turn-in directory: <i>ex07/</i>	
Files to turn in: <i>ft_match.rs</i>	
Allowed functions: <i>print!()</i> , <i>println!()</i>	

The **match** keyword in Rust is extremely powerful and must be exhaustive (all possible cases must be handled).

Write a program that declares a variable containing the value 2. Use a **match** statement to evaluate it and print:

- One if the value is 1;
- Two if the value is 2;
- Three if the value is 3;
- Other for any other value.

Expected output:

```
$> ./ft_match
Two
```



The `_` pattern acts as a default case. It matches anything that hasn't been specified in the previous arms.

Chapter XI

Turn in and Submission

Turn in and Submission

Turn in all your files in a Git repository as usual. All files in your repository will be evaluated. Double-check your file names and make sure all required files are present in the repository.



You must only turn in the files requested by the subject. Make sure you understand the Rust basics and be ready to correctly explain your code during evaluation.